

Q1. Objective Questions Answers

(I) Which of the following is NOT a valid aggregate function in SQL?

✓ Answer: (D) MINUS

- SUM, AVG, and COUNT are aggregate functions.
 - MINUS is a set operator, not an aggregate function.
-

(II) Which JOIN returns all rows from both tables with NULL where there is no match?

✓ Answer: (D) FULL OUTER JOIN

- It returns all matching and non-matching rows from both tables.
-

(III) Which ensures that decomposition does not lose information?

✓ Answer: (A) Lossless join

- Lossless join decomposition preserves all original information.
-

(IV) What kind of schedule always preserves serializability?

✓ Answer: (A) Conflict Serializable schedule

- Conflict serializable schedules are guaranteed to be equivalent to serial schedules.
-

(V) Which concurrency control method uses system time to order transactions?

✓ Answer: (C) Timestamp ordering

- Timestamp ordering assigns timestamps to transactions and executes them accordingly.
-

Q2. Write an SQL query for:

[CO3, 13] [5]

(a) Write a query to get employees who belong to departments with less than 3 employees,

(b) How to find the top 3 highest-paid employees in each department?

(c) Write a query to find all employees who earn more than the average salary of their department. Assume the table name is Employee with columns:

- emp_id
- emp_name
- department_id
- salary

(a) Employees who belong to departments with less than 3 employees

```
SELECT *  
FROM Employee  
WHERE department_id IN (  
    SELECT department_id  
    FROM Employee
```

```

GROUP BY department_id
HAVING COUNT(*) < 3
);

```

(b) Top 3 highest-paid employees in each department

```

SELECT emp_id, emp_name, department_id, salary
FROM (
    SELECT emp_id,
           emp_name,
           department_id,
           salary,
           DENSE_RANK() OVER (
               PARTITION BY department_id
               ORDER BY salary DESC
           ) AS rnk
    FROM Employee
) AS ranked
WHERE rnk <= 3;

```

(c) Employees who earn more than the average salary of their department

```

SELECT e.emp_id,
       e.emp_name,
       e.department_id,
       e.salary
FROM Employee e
WHERE e.salary > (
    SELECT AVG(salary)
    FROM Employee
    WHERE department_id = e.department_id
);

```

Comparison between Two-Phase Locking (2PL) and Timestamp Ordering

Feature	Two-Phase Locking (2PL)	Timestamp Ordering
Definition	A concurrency control method where transactions use locks before accessing data.	A method where transactions are ordered based on timestamps assigned at their start time.
Serializability	Ensures conflict serializability by following locking rules.	Ensures serializability using timestamp order of transactions.
Working Principle	Transaction has two phases: Growing phase (acquire locks) and Shrinking phase (release locks).	Operations are executed according to transaction timestamps. Older transactions get priority.
Deadlock Possibility	Deadlocks can occur because transactions may wait for locks held by others.	No deadlock occurs because transactions do not wait; conflicting transactions are rolled back.
Starvation	Less chance of starvation.	Starvation may occur due to repeated rollbacks of younger transactions.
Blocking	Transactions may block while waiting for locks.	Non-blocking protocol; transactions execute without waiting.

Feature	Two-Phase Locking (2PL)	Timestamp Ordering
Efficiency	Efficient in systems with moderate contention, but locking overhead exists.	Efficient for read-heavy systems, but excessive rollbacks reduce performance under high contention.
Complexity	Easier to understand and implement.	More complex because timestamps must be maintained for each transaction and data item.
Data Consistency	Maintains data consistency using locks.	Maintains consistency through timestamp validation.
Recovery Handling	Easier recovery due to lock-based control.	Recovery can be difficult because aborted transactions may need repeated execution.
Conclusion • 2PL is widely used because it guarantees serializability and maintains consistency, but it may cause deadlocks. • Timestamp Ordering avoids deadlocks and allows non-blocking execution, but it may suffer from frequent rollbacks and starvation.		

Q3. Compare and contrast the Two-Phase Locking (2PL) and Timestamp Ordering concurrency control protocols in terms of serializability, deadlock possibility, and efficiency. (a) Relation: R(M, N, O, P, Q)

Given Functional Dependencies:

1. $(MN \rightarrow O)$
2. $(O \rightarrow P)$
3. $(P \rightarrow Q)$

Step 1: Find Closure of (MN)

```
[
(MN)+ = {M, N}
]
Using (MN → O)
[
(MN)+ = {M, N, O}
]
Using (O → P)
[
(MN)+ = {M, N, O, P}
]
Using (P → Q)
[
(MN)+ = {M, N, O, P, Q}
]
```

So, (MN) determines all attributes.

Step 2: Check Minimality

- $(M)^+ = \{M\}$
- $(N)^+ = \{N\}$

Neither alone can determine all attributes.

Therefore, **MN is minimal.**

Candidate Key

[
 $\boxed{MN}$
]

(b) Relation: R(A, B, C, D)

Given Functional Dependencies:

1. $A \rightarrow B$
 2. $BC \rightarrow D$
 3. $D \rightarrow A$
-

Step 1: Find Closure of (AC)

[
 $(AC)^+ = \{A, C\}$
]

Using $A \rightarrow B$

[
 $(AC)^+ = \{A, B, C\}$
]

Using $BC \rightarrow D$

[
 $(AC)^+ = \{A, B, C, D\}$
]

So, (AC) is a super key.

Check minimality:

- $A^+ = \{A, B\}$
- $C^+ = \{C\}$

Neither determines all attributes.

Therefore, (AC) is a candidate key.

Step 2: Find Closure of (BC)

[
 $(BC)^+ = \{B, C\}$
]

Using $BC \rightarrow D$

[
 $(BC)^+ = \{B, C, D\}$
]

Using $D \rightarrow A$

[
 $(BC)^+ = \{A, B, C, D\}$
]

Thus, (BC) is also a candidate key.

Step 3: Find Closure of (CD)

```
[
(CD)^+ = {C, D}
]
Using (D  $\rightarrow$  A)
[
(CD)^+ = {A, C, D}
]
Using (A  $\rightarrow$  B)
[
(CD)^+ = {A, B, C, D}
]
Thus, (CD) is also a candidate key.
```

Final Answer

(a)

Candidate Key:

```
[
\boxed{MN}
]
```

(b)

Candidate Keys:

```
[
\boxed{AC, \ BC, \ CD}
]
```

Q5. A database application requires high throughput and minimal locking delays, even at the cost of occasional inconsistencies in non-critical data.

Which concurrency controls strategy would you recommend, and why? Justify your answer with reference

to different concurrency control mechanisms and their trade-offs.

5 Recommended Concurrency Control Strategy

For a database application that requires:

- **High throughput**
- **Minimal locking delays**
- Can tolerate **occasional inconsistencies in non-critical data**

the best choice is:

Timestamp Ordering or Optimistic Concurrency Control (OCC)

Justification

Traditional locking methods like **Two-Phase Locking (2PL)** ensure strong consistency but reduce performance because transactions must wait for locks.

In this case, performance and speed are more important than strict consistency, so non-locking or low-locking methods are preferable.

Comparison of Concurrency Control Mechanisms

Mechanism	Advantages	Disadvantages	Suitability
Two-Phase Locking (2PL)	Guarantees serializability and consistency	Causes blocking and deadlocks due to locks	Not suitable because locking delays reduce throughput
Strict 2PL	Prevents cascading rollback and ensures recoverability	High waiting time and reduced concurrency	Poor choice for high-speed systems
Timestamp Ordering	Deadlock-free and non-blocking	Some transactions may rollback frequently	Good choice for high throughput
Optimistic Concurrency Control (OCC)	No locks during execution, very high concurrency	Validation failures may cause transaction restart	Best for low-conflict environments
Multiversion Concurrency Control (MVCC)	Readers and writers do not block each other	Requires extra storage for versions	Excellent for read-heavy applications

Recommended Approach

1. Optimistic Concurrency Control (Preferred)

Why?

- Transactions execute freely without locking resources.
- No waiting time for locks.
- High transaction throughput.
- Suitable when conflicts are rare or data is non-critical.

Trade-off

- Some transactions may abort and restart if conflicts occur.
- Temporary inconsistencies are possible.

Since the application accepts occasional inconsistencies, this trade-off is acceptable.

2. Timestamp Ordering (Alternative)

Why?

- Transactions are ordered using timestamps.
- No deadlocks because transactions never wait.
- Faster than lock-based protocols.

Trade-off

- Older transactions get priority.
- Frequent rollbacks may happen under heavy contention.

Still better than locking systems when throughput is more important.

Final Conclusion

For this application, **Optimistic Concurrency Control (OCC)** is the most suitable strategy because it provides:

- Very high concurrency
- Minimal locking overhead
- Better throughput
- No deadlocks

Although occasional inconsistencies and transaction rollbacks may occur, these are acceptable because the data is non-critical.

If the system is highly read-intensive, **MVCC** can also be combined to further improve performance.

(a) Relation: R(A, B, C, D, E)

Q6. Find the canonical cover of the following set of functional dependencies.

[CO4, L4] [5]

(a) Given: R (A, B, C, D, E), FD= (AB-C, A-C, C-D, D-E)

(b) Given: R (W, X, Y, Z); FD= (WX-YZ, W-X, X-Y)

Given Functional Dependencies:

1. $(AB \rightarrow C)$
2. $(A \rightarrow C)$
3. $(C \rightarrow D)$
4. $(D \rightarrow E)$

Step 1: Remove Extraneous Attributes

Compare:

- $(AB \rightarrow C)$
- $(A \rightarrow C)$

Since (A) alone determines (C), attribute (B) is extraneous in $(AB \rightarrow C)$.

Therefore:

[
 $AB \rightarrow C$
]

is redundant and can be removed.

Remaining FDs:

1. $(A \rightarrow C)$
2. $(C \rightarrow D)$
3. $(D \rightarrow E)$

Step 2: Check for Redundant Dependencies

None of the remaining FDs can be derived from others.

So no more reduction is possible.

Canonical Cover

[
 $\boxed{\{A \rightarrow C, C \rightarrow D, D \rightarrow E\}}$
]

(b) Relation: R(W, X, Y, Z)

Given Functional Dependencies:

1. $(WX \rightarrow YZ)$
2. $(W \rightarrow X)$
3. $(X \rightarrow Y)$

Step 1: Split Right-Hand Side

Split $(WX \rightarrow YZ)$ into:

1. $(WX \rightarrow Y)$
2. $(WX \rightarrow Z)$

Now FDs become:

1. $(WX \rightarrow Y)$
 2. $(WX \rightarrow Z)$
 3. $(W \rightarrow X)$
 4. $(X \rightarrow Y)$
-

Step 2: Remove Redundant Dependencies**Check $(WX \rightarrow Y)$**

From:

[
 $W \rightarrow X$
]

and

[
 $X \rightarrow Y$
]

we get:

[
 $W \rightarrow Y$
]

Hence $(WX \rightarrow Y)$ is redundant.

Remove it.

Remaining:

1. $(WX \rightarrow Z)$
 2. $(W \rightarrow X)$
 3. $(X \rightarrow Y)$
-

Step 3: Check Extraneous Attributes in $(WX \rightarrow Z)$

Since:

[
 $W \rightarrow X$
]

then (W) already determines (X) .

Therefore (X) is extraneous in $(WX \rightarrow Z)$.

So:

[
 $WX \rightarrow Z$
]

becomes:

[
 $W \rightarrow Z$
]

Canonical Cover

[
 $\boxed{\{W \rightarrow X, X \rightarrow Y, W \rightarrow Z\}}$
]

Q7. Find Highest Normalization for-

(a) Given: R (A, B, C, D, E, F): FD= (AB→C, C→DE, E,F→A) Given Relation

[
R(A, B, C, D, E, F)
]

Functional Dependencies:

1. $AB \rightarrow C$
2. $C \rightarrow DE$
3. $EF \rightarrow A$

Step 1: Find Candidate Key

Compute closure of (ABF):

- $AB \rightarrow C$
- $C \rightarrow DE$
- Using (E) and (F), $EF \rightarrow A$

Thus:

[
 $(ABF)^{++} = \{A, B, C, D, E, F\}$
]

So, **ABF is a candidate key.**

Prime attributes:

[
A, B, F
]

Non-prime attributes:

[
C, D, E
]

Step 2: Check Normal Forms

First Normal Form (1NF)

Relation contains atomic attributes.

✓ Satisfies 1NF.

Second Normal Form (2NF)

A relation is in 2NF if:

- It is in 1NF
- No non-prime attribute depends on a proper subset of candidate key.

Candidate key = (ABF)

Check dependencies:

- $AB \rightarrow C$

Here, (AB) is a proper subset of candidate key (ABF), and (C) is non-prime.
So, **partial dependency exists**.

✗ Not in 2NF.

Highest Normal Form

Since the relation satisfies only 1NF and violates 2NF:

[
 $\boxed{\text{Highest Normal Form} = 1\text{NF}}$
]